

# Digital Image Processing

## Lab 1 2021-3-31

- ✓ Get to Know the Environment
- ✓ Get started with IPT
- ✓ Image Histograms

### Matlab programming for digital image processing

For the labs we will use Matlab (short for “Matrix Laboratory”), a large and very powerful software tool for numeric computation, data analysis and signal processing. It features a high-level programming language and an interactive graphical environment for algorithm development and visualization. You will find links to tutorials on the course web page.

Using a web browser, go to the homepage of the course book at <http://www.imageprocessingplace.com/> and download the “Standard” test images (a set of images found frequently in the literature: Lena, peppers, cameraman, lake, etc.). Try out the basic MATLAB functions for loading, saving, displaying and manipulating digital images on the test images. A few test images can also be found on the course web page.

## 1 Review the Matlab Environment

After starting the MATLAB application, you will get a “desktop” with several sub- windows. Try the following example, using functions from the Image Processing toolbox, on images from the course Web page:

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Introducing Matlab (adapted from http://www.cns.nyu.edu/~eero and
% http://www.cs.dartmouth.edu/~farid/teaching/cs88/matlab.intro.html)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% (1) Help and basics

% The symbol "%" is used in front of a comment.

% To get help type "help" (will give list of help topics) or "help topic"

% If you don't know the exact name of the topic or command you are looking for,
% type "lookfor keyword" (e.g., "lookfor regression")

% When writing a long matlab statement that exceeds a single row use ...
% to continue statement to next row.
```

% When using the command line, a ";" at the end means matlab will not display the result. If ";" is omitted then matlab will display result.

% Use the up-arrow to recall commands without retyping them (and down arrow to go forward in commands).

% C-a moves to beginning of line (C-e for end), C-f moves forward a character (C-b moves back), C-d deletes a character, C-k deletes the line to the right of the cursor, C-p goes back through the command history and C-n goes forward (equivalent to up and down arrows), % tab command completion.

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% (2) Objects in matlab -- the basic objects in matlab are scalars, % vectors, and matrices...

|   |                  |                                                              |
|---|------------------|--------------------------------------------------------------|
| N | = 5              | % a scalar                                                   |
| v | = [1 0 0]        | % a row vector                                               |
| v | = [1;2;3]        | % a column vector                                            |
| v | = v'             | % transpose a vector<br>(row to column or column to row)     |
| v | = [1:5:3]        | % a vector in a specified range:                             |
| v | = pi*[-4:4]/4    | % [start:end] or [start:stepsize:end]                        |
| v | = []             | % empty vector                                               |
|   |                  |                                                              |
| m | = [1 2 3; 4 5 6] | % a matrix: 1ST parameter is ROWS<br>% 2ND parameter is COLS |
| m | = zeros(2,3)     | % a matrix of zeros                                          |
| v | = ones(1,3)      | % a matrix of ones                                           |
| m | = eye(3)         | % identity matrix                                            |
| v | = rand(3,1)      | % random matrix with values in [0,1] (see also randn)        |

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

|        |                  |                                                                 |
|--------|------------------|-----------------------------------------------------------------|
| m      | = [1 2 3; 4 5 6] |                                                                 |
| m(1,3) |                  | % access a matrix element<br>% matrix (rownumber, columnnumber) |
| m(2,:) |                  | % access a matrix row (2nd row)                                 |
| m(:,1) |                  | % access a matrix column (1st row)                              |

|           |                     |
|-----------|---------------------|
| size(m)   | % size of a matrix  |
| size(m,1) | % number rows       |
| size(m,2) | % number of columns |

```

m1= zeros(size(m))           % create a new matrix with size of m
who                           % list of variables
whos                           % list/size/type of variables

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

% (3) Simple operations on vectors and matrices

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

% (A) Pointwise (element by element) Operations:

```

```

% addition of vectors/matrices and multiplication by a scalar
% are done "element by element"

```

```

a= [1 2 3 4];                % vector
2 * a                         % scalar multiplication
a / 4                          % scalar multiplication
b= [5 6 7 8];                % vector
a + b                          % pointwise vector addition
a - b                          % pointwise vector addition
a .^ 2                         % pointwise vector squaring (note .)
a .* b                         % pointwise vector multiply (note .)
a ./ b                         % pointwise vector divide (note .)

```

```

log( [1 2 3 4] )              % pointwise arithmetic operation
round( [1.5 2; 2.2 3.1] )      % pointwise arithmetic operation

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

% (B) Vector Operations (no for loops needed)

```

```

% Built-in matlab functions operate on vectors, if a matrix is given,
% then the function operates on each column of the matrix

```

```

a= [1 4 6 3]                  % vector
sum(a)                         % sum of vector elements
mean(a)                        % mean of vector elements
var(a)                         % variance
std(a)                         % standard deviation
max(a)                         % maximum

```

```

a      = [1 2 3; 4 5 6]        % matrix
a(:)    % vectorized version of the matrix
mean(a)  % mean of each column
max(a)    % max of each column
max(max(a)) % to obtain max of matrix
max(a(:)) % or...

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

## % (C) Matrix Operations:

`[1 2 3] * [4 5 6]'`                      % row vector 1x3 times column vector 3x1  
                                              % results in single number, also  
                                              % known as dot product or inner product

`[1 2 3]' * [4 5 6]`                      % column vector 3x1 times row vector 1x3  
                                              % results in 3x3 matrix, also  
                                              % known as outer product

`a= rand(3,2)`                              % 3x2 matrix  
`b= rand(2,4)`                              % 2x4 matrix  
`c= a * b`                                    % 3x4 matrix

`a`                = `[1 2; 3 4; 5 6]`              % 3 x 2 matrix  
`b`                = `[5 6 7];`                      % 1 x 3 vector  
`b * a`                                      % matrix multiply  
`a' * b'`                                      % matrix multiply

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

## % (4) Saving your work

`save mysession`                              % creates mysession.mat with all variables  
`save mysession a b`                          % save only variables a and b

`clear all`                                    % clear all variables  
`clear a b`                                    % clear variables a and b

`load mysession`                              % load session  
`a`  
`b`

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%  
 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

## % (5) Relations and control statements

% Example: given a vector `v`, create a new vector with values equal to  
 % `v` if they are greater than 0, and equal to 0 if they less than or equal to 0.

`v`                = `[3 5 -2 5 -1 0]`              % 1: FOR LOOPS  
`u`                = `zeros( size(v) );`            % initialize  
`for i = 1:size(v,2)`                      % size(v,2) is the number of columns  
                   `if( v(i) > 0 )`  
                                              `u(i) = v(i);`  
                   `end`  
`end`  
`u`

```

v      = [3 5 -2 5 -1 0]          % 2: NO FOR LOOPS
u2     = zeros( size(v) );        % initialize
ind    = find( v>0 )              % index into >0 elements
u2(ind) = v( ind )

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

%(6) Creating functions using m-files:

% Functions in matlab are written in m-files. Create a file called  
% 'thres.m' In this file put the following 4 lines:

```

function res = thres( v )
u= zeros( size(v) );          % initialize
ind= find( v>0 )              % index into >0 elements
u(ind)= v( ind )
v= [3 5 -2 5 -1 0]
thres( v )                    % call from command line

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

%(7) Plotting

```

x= [0 1 2 3 4];              % basic plotting
plot( x );
plot( x, 2*x );
axis( [0 8 0 8] );

```

```

x= pi*[-24:24]/24;
plot( x, sin(x) );
xlabel( 'radians' );
ylabel( 'sin value' );
title( 'dummy' );
gtext( 'put cursor where you want text and press mouse' );

```

```

figure;                        % multiple functions in separate graphs
subplot( 1,2,1 );
plot( x, sin(x) );
axis square;
subplot( 1,2,2 );
plot( x, 2.*cos(x) );
axis square;

```

```

figure;                        % multiple functions in single graph
plot( x,sin(x) );
hold on;                       % hold on tells matlab to write on top
plot( x, 2.*cos(x), '--' );    % of the current plot
legend( 'sin', 'cos' );
hold off;

```

```
figure;                                % matrices as images
m = rand(64,64);
imagesc(m)
colormap gray;
axis image
axis off;
```

## 2. Get started with IPT

Matlab->Help->Product help-> Image Processing Toolbox-> Getting started->

### 1) Reading and Displaying Images

- **Search** imread, imshow, imtool

Example:

```
I=imread('cameraman.tif');
imtool(I);
imshow(I);
```

### 2) Displaying Image Information

- **Search** whos, size, class

Example:

```
I=imread('cameraman.tif');
whos I
size (I)
class (I)
```

### 3) Converting Between Different Formats

- **Search** gray2ind, ind2gray, ind2rgb, mat2gray, rgb2gray, rgb2ind, im2bw, dither

Example:

```
I=imread('onion.png');
I_bin=im2bw(I);
I_gray=rgb2gray(I);
[I_ind,map]=rgb2ind(I,8);

map_custom=[
    1 1 1; %white
    1 0 0; %red
    0 0 0]; %black
I_cus=dither(I,map_custom);
figure,imshow(I_bin)
figure,imshow(I_gray)
figure,imshow(I_ind,map)
figure,imshow(I_cus,map_custom)
```

### 4) Saving Images

- **Search** imwrite

Example:

```
imwrite(I_bin, 'onion_bin.jpg')
imwrite(I_cus, map_custom, 'onion.jpg')
```

### 3. Intensity transformations

#### 1) Image Negatives

- **Search** imcomplement

Example:

```
I = imread('cameraman.tif');
J = imcomplement(I);
imshow(J)
```

#### 2) Log transformation **Using** $C \cdot \log(1+r)$

Example:

```
I = imread('cameraman.tif');
imshow(I)
I2=im2double(I);
J=1*log(1+I2);
J2=2*log(1+I2);
J3=5*log(1+I2);
figure, imshow(J)
figure, imshow(J2)
figure, imshow(J3)
```

#### 3) Gamma transformation

- **Search** imadjust

Example:

```
I = imread('cameraman.tif');
J=imadjust(I, [], [], 1);
J2=imadjust(I, [], [], 3);
J3=imadjust(I, [], [], 0.4);
figure, imshow(J);
figure, imshow(J2);
figure, imshow(J3);
```

#### 4) Contrast-stretching transformations **Using** $1./(1+(m./(double(f)+eps)).^E)$

Example:

```
I = imread('cameraman.tif');
I2=im2double(I);
m=mean2(I2);
contrast1=1./(1+(m./(I2+eps)).^4);
contrast2=1./(1+(m./(I2+eps)).^5);
contrast3=1./(1+(m./(I2+eps)).^10);
imshow(I2)
figure,imshow(contrast1)
figure,imshow(contrast2)
figure,imshow(contrast3)|
```

## 4. Image Histogram

Matlab->Help->Product help->

- **Search** imhist
- **Search** histeq

Example:

```
h_cameraman=imread('cameraman.tif');
figure,imhist(h_cameraman);
title('Histogram of cameraman.tif');
```

```
%Histogram Equalization
cameraman_eq=histeq(h_cameraman);
figure, imshow(cameraman_eq);
figure, imhist(cameraman_eq);
```

- **Search** imhistmatch

Example:

```
%Histogram Matching
J=imhistmatch(cameraman_eq,h_cameraman);
figure, imshow(J);
figure, imhist(J);
```